Lecture 04
Model-based Recommender Systems (part 3)

Euhyun Moon, Ph.D.
Machine Learning System (MLSys) Lab
Computer Science and Engineering
Sogang University



Today's Class

- Model-based Recommender Systems
  - Advanced Latent Factor Models for Implicit Feedbacks

Word Embedding

- Words in dictionary: {Monkey, Lion}



Word Embedding

- Words in dictionary: {Monkey, Lion}



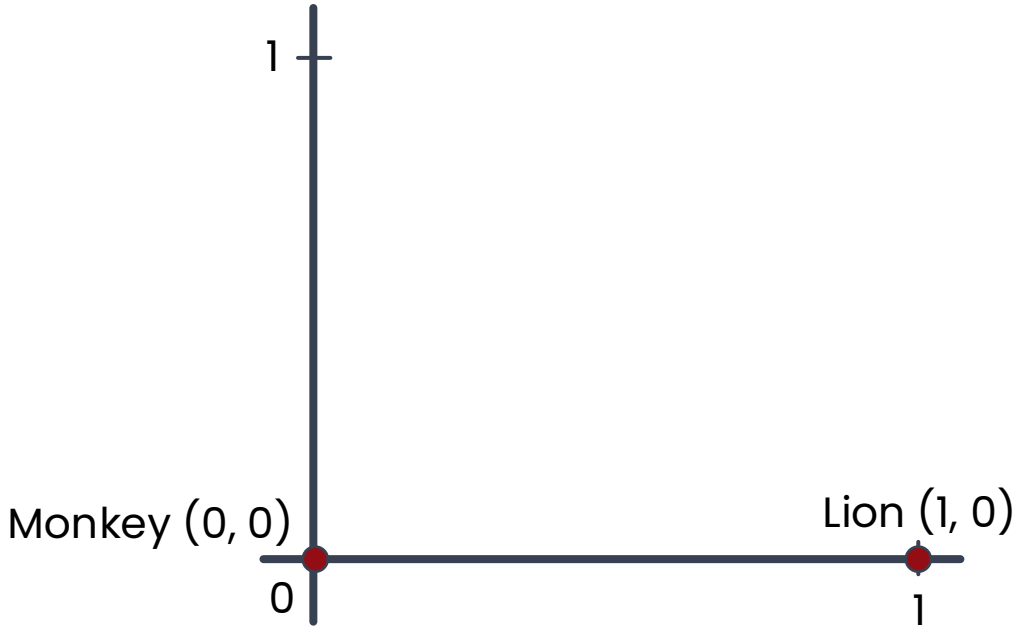
Word Embedding

- Words in dictionary: {Monkey, Lion, Apple, Banana}



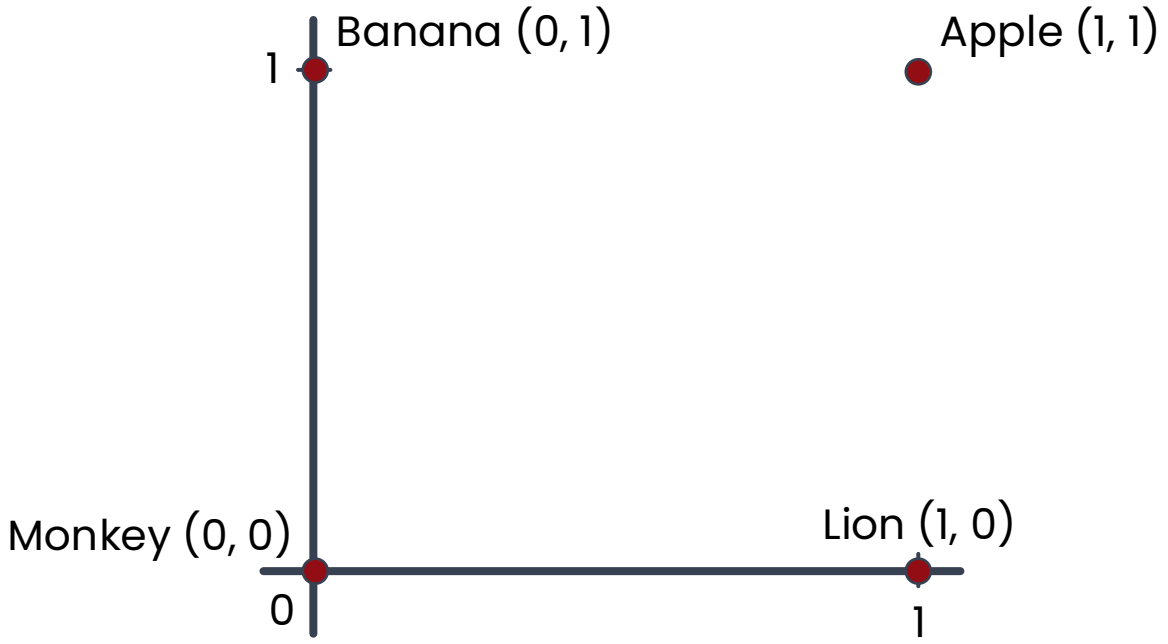
Word Embedding

- Words in dictionary: {Monkey, Lion, **Apple, Banana**}



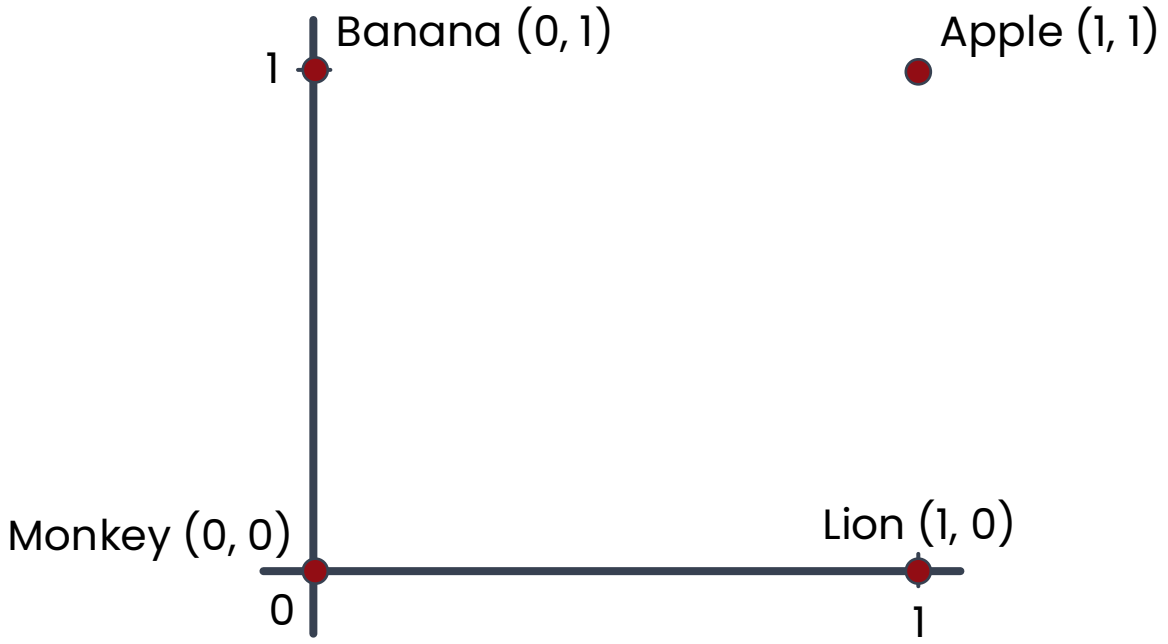
Word Embedding

- Words in dictionary: {Monkey, Lion, **Apple, Banana**}



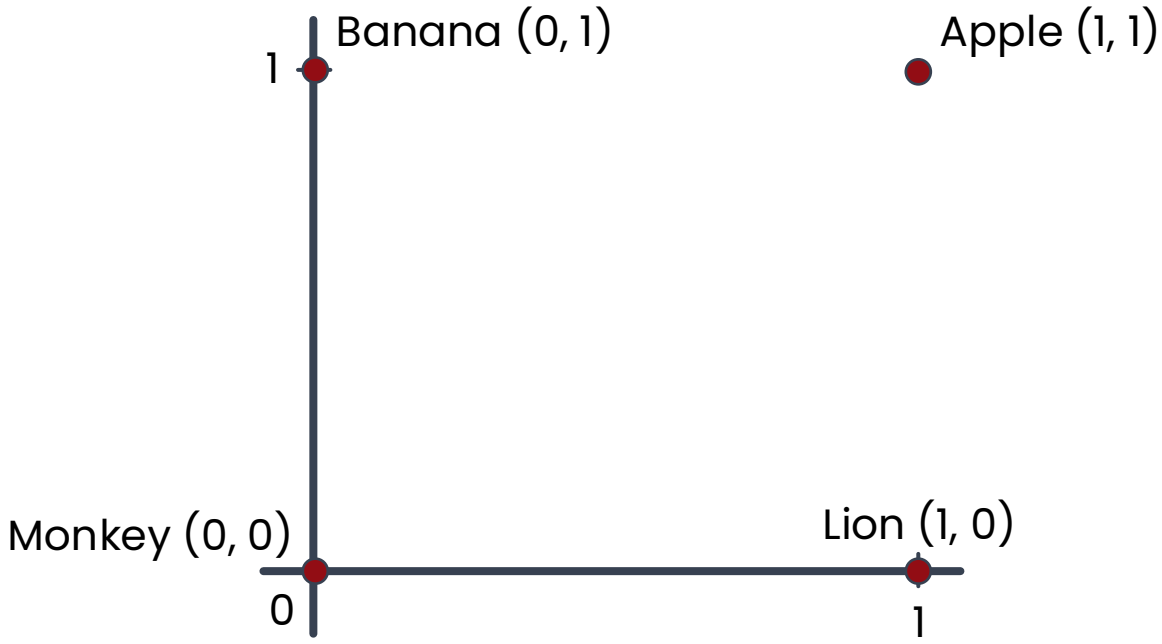
Word Embedding

- Q: Which fruit do monkeys prefer more: apples or bananas?



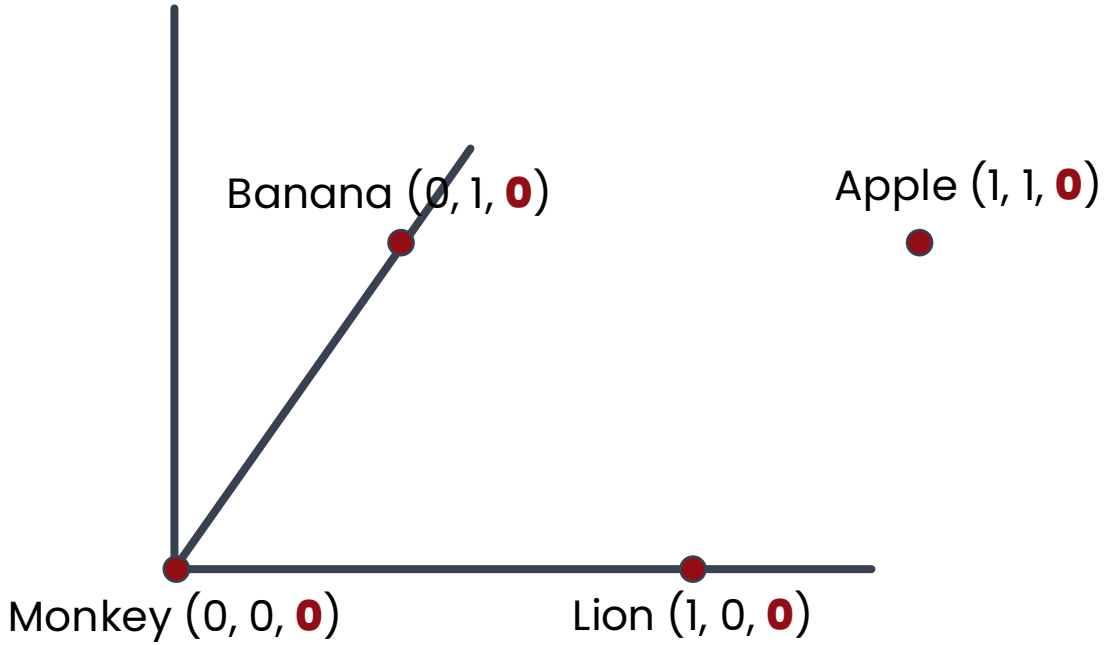
Word Embedding

- What if we want to represent the words **human** and **robot**?



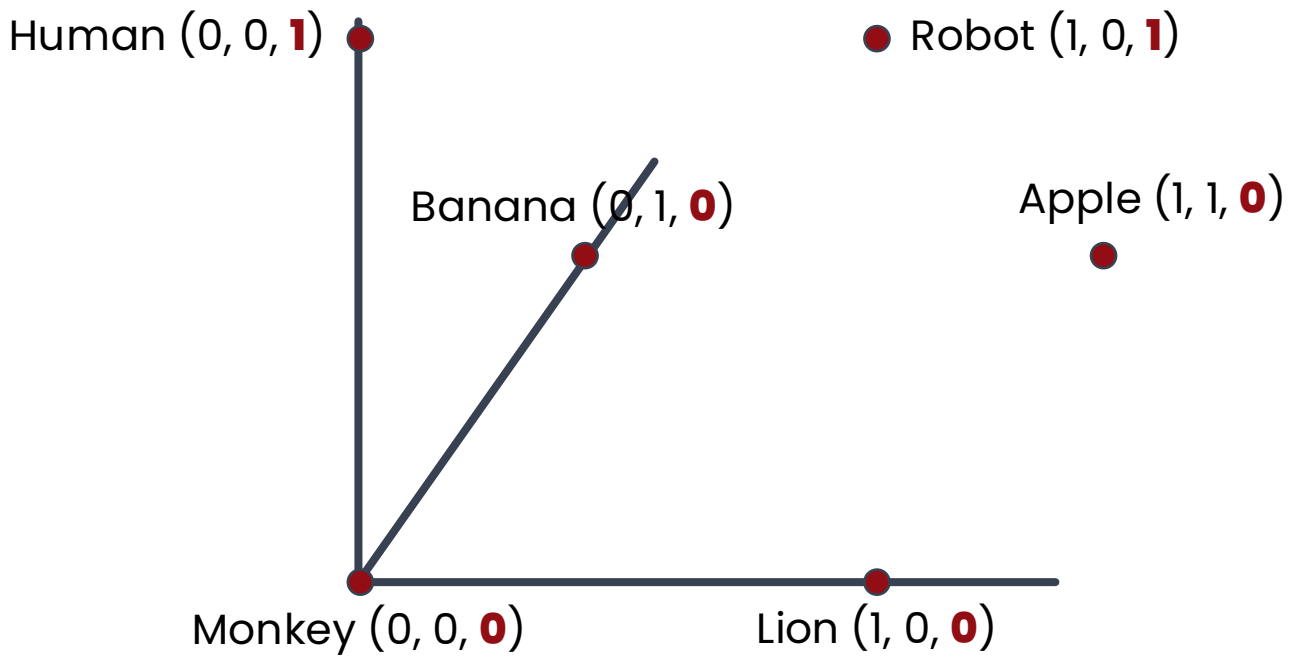
Word Embedding

- Extending the dimensionality or rank of the latent vector space



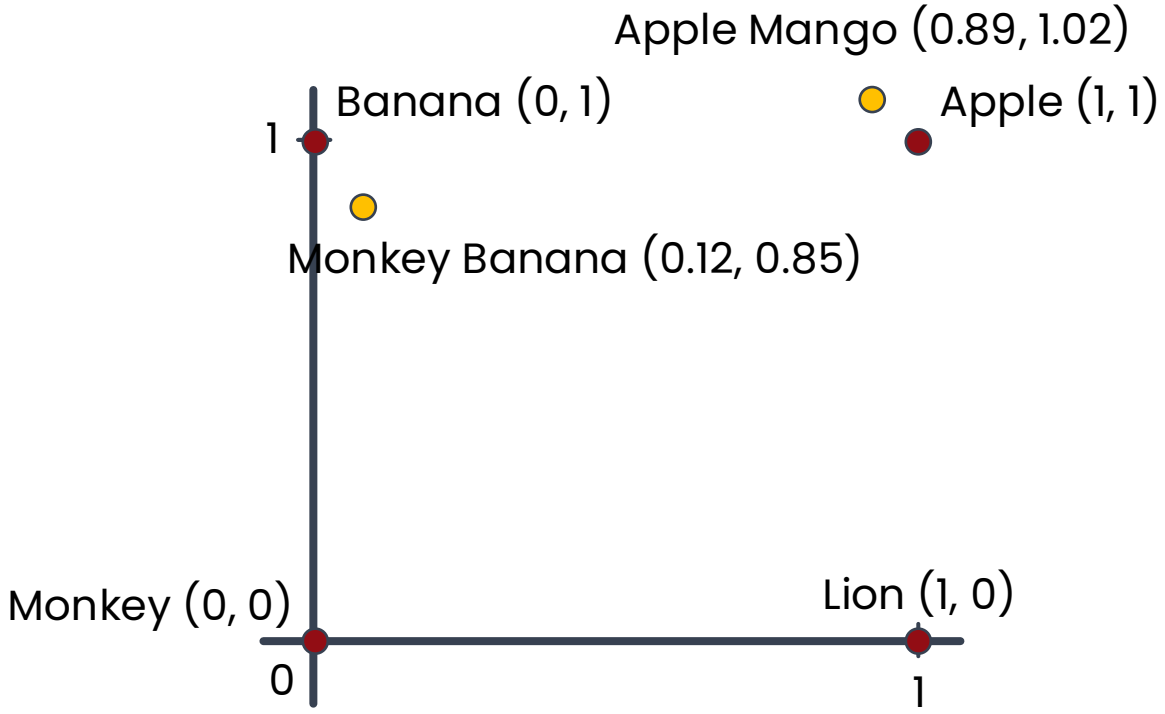
Word Embedding

- Extending the dimensionality or rank of the latent vector space

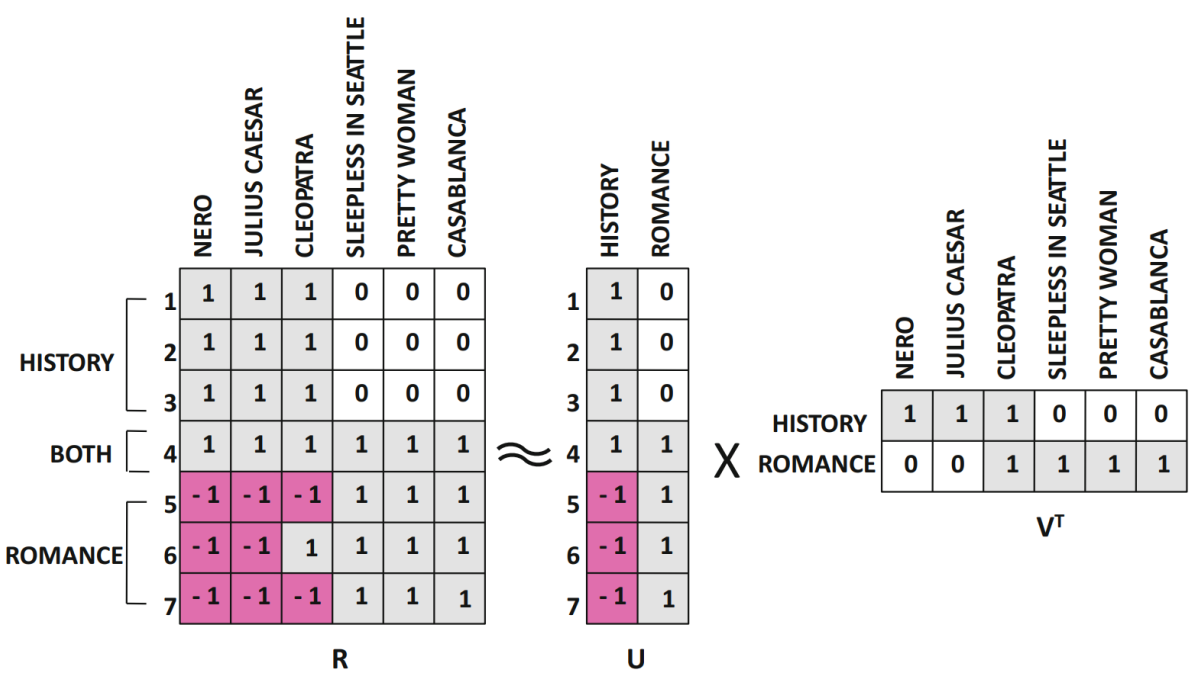


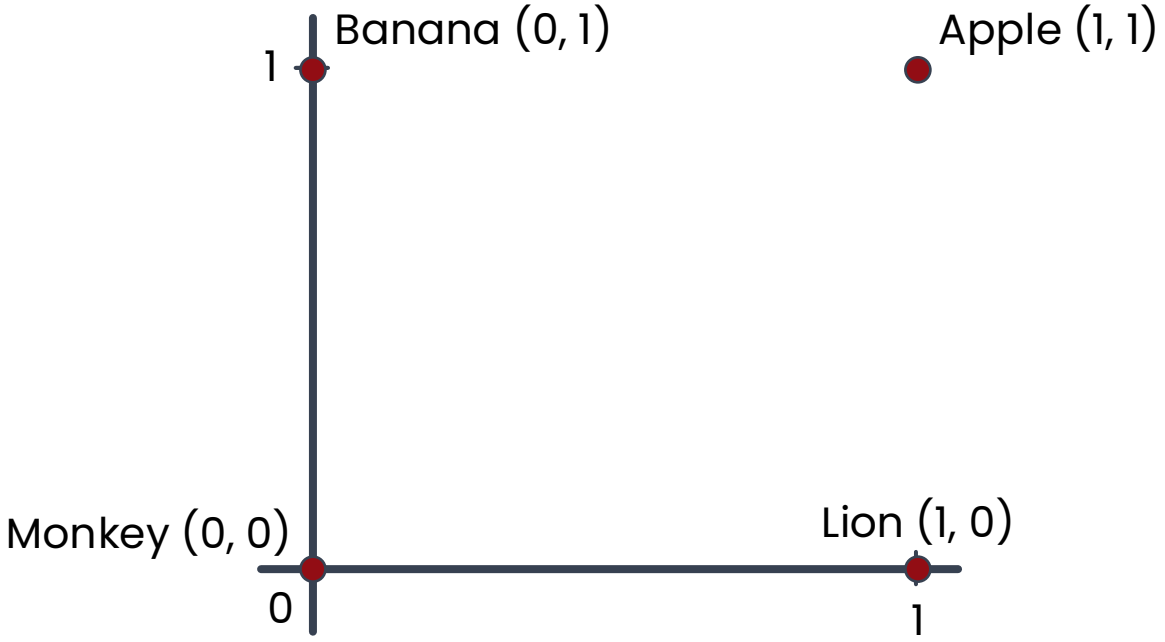
Word Embedding

- In fact, we can use continuous values to represent words



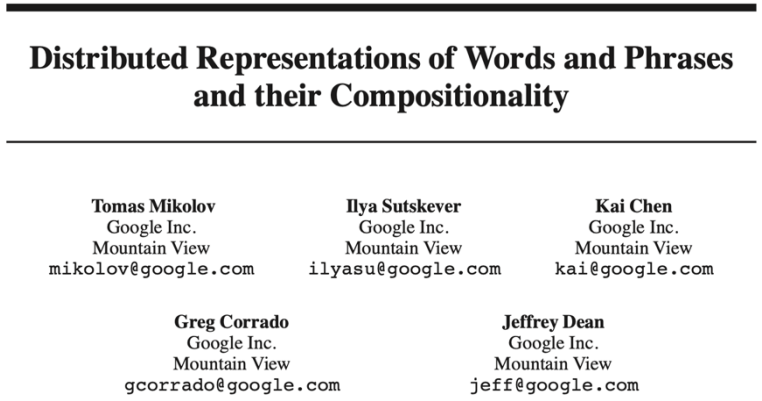
Word Embedding





Word2Vec

- Distributed representations of words and phrases and their compositionality, Mikolov et al. NeurIPS13



Distributed Representations of Words and Phrases and their Compositionality

Tomas Mikolov
Google Inc.
Mountain View
mikolov@google.com

Ilya Sutskever
Google Inc.
Mountain View
ilyasu@google.com

Kai Chen
Google Inc.
Mountain View
kai@google.com

Greg Corrado
Google Inc.
Mountain View
gcorrado@google.com

Jeffrey Dean
Google Inc.
Mountain View
jeff@google.com

Distributed representations of words and phrases and their compositionality

T Mikolov, I Sutskever, K Chen, GS Corrado, J Dean

Advances in neural information processing systems, 2013 • proceedings.neurips.cc

Abstract

The recently introduced continuous Skip-gram model is an efficient method for learning high-quality distributed vector representations that capture a large number of precise syntactic and semantic word relationships. In this paper we present several improvements that make the Skip-gram model more expressive and enable it to learn higher quality vectors more rapidly. We show that by subsampling frequent words we obtain significant speedup, and also learn higher quality representations as measured by our tasks. We also

자세히 보기

☆ 저장 99 인용 45396회 인용 관련 학술자료 전체 55개의 버전 »



Distributed Representations of Words and Phrases and their Compositionality

Tomas Mikolov
Google Inc.
Mountain View
mikolov@google.com

Ilya Sutskever
Google Inc.
Mountain View
ilyasu@google.com

Kai Chen
Google Inc.
Mountain View
kai@google.com

Greg Corrado
Google Inc.
Mountain View
gcorrado@google.com

Jeffrey Dean
Google Inc.
Mountain View
jeff@google.com

Abstract

The recently introduced continuous Skip-gram model is an efficient method for learning high-quality distributed vector representations that capture a large number of precise syntactic and semantic word relationships. In this paper we present several extensions that improve both the quality of the vectors and the training speed. By subsampling of the frequent words we obtain significant speedup and also learn more regular word representations. We also describe a simple alternative to the hierarchical softmax called negative sampling.

An inherent limitation of word representations is their indifference to word order and their inability to represent idiomatic phrases. For example, the meanings of "Canada" and "Air" cannot be easily combined to obtain "Air Canada". Motivated by this example, we present a simple method for finding phrases in text, and show that learning good vector representations for millions of phrases is possible.

1 Introduction

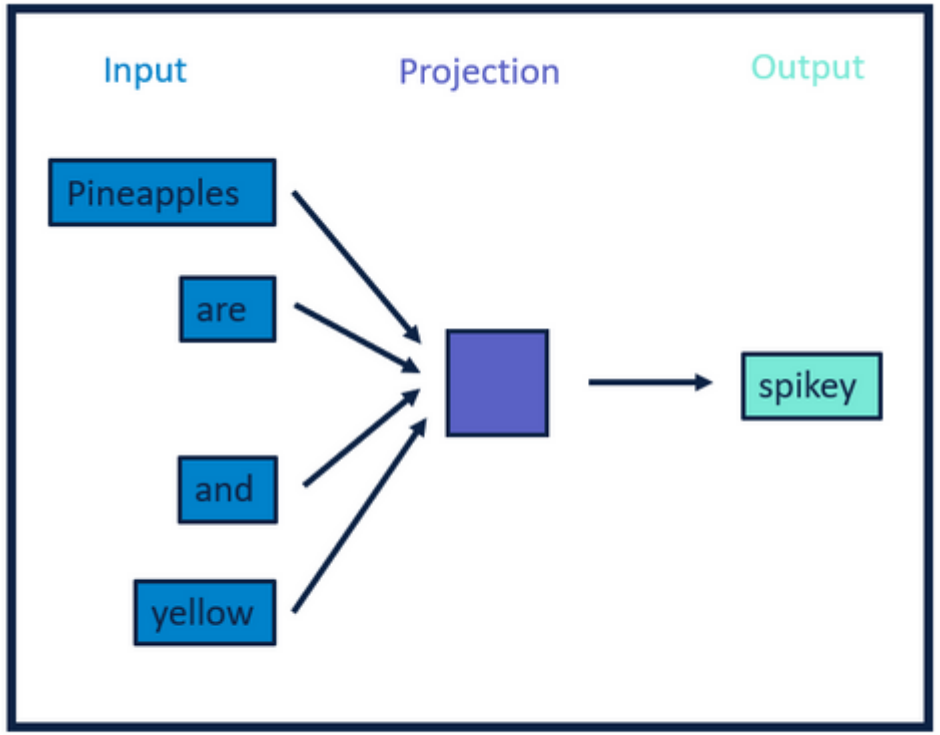
Distributed representations of words in a vector space help learning algorithms to achieve better performance in natural language processing tasks by grouping similar words. One of the earliest use of word representations dates back to 1986 due to Rumelhart, Hinton, and Williams [13]. This idea has since been applied to statistical language modeling with considerable success [1]. The follow up work includes applications to automatic speech recognition and machine translation [14, 7], and a wide range of NLP tasks [2, 20, 15, 3, 18, 19, 9].

Recently, Mikolov et al. [8] introduced the Skip-gram model, an efficient method for learning high-quality vector representations of words from large amounts of unstructured text data. Unlike most of the previously used neural network architectures for learning word vectors, training of the Skip-gram model (see Figure 1) does not involve dense matrix multiplications. This makes the training extremely efficient: an optimized single-machine implementation can train on more than 100 billion words in one day.

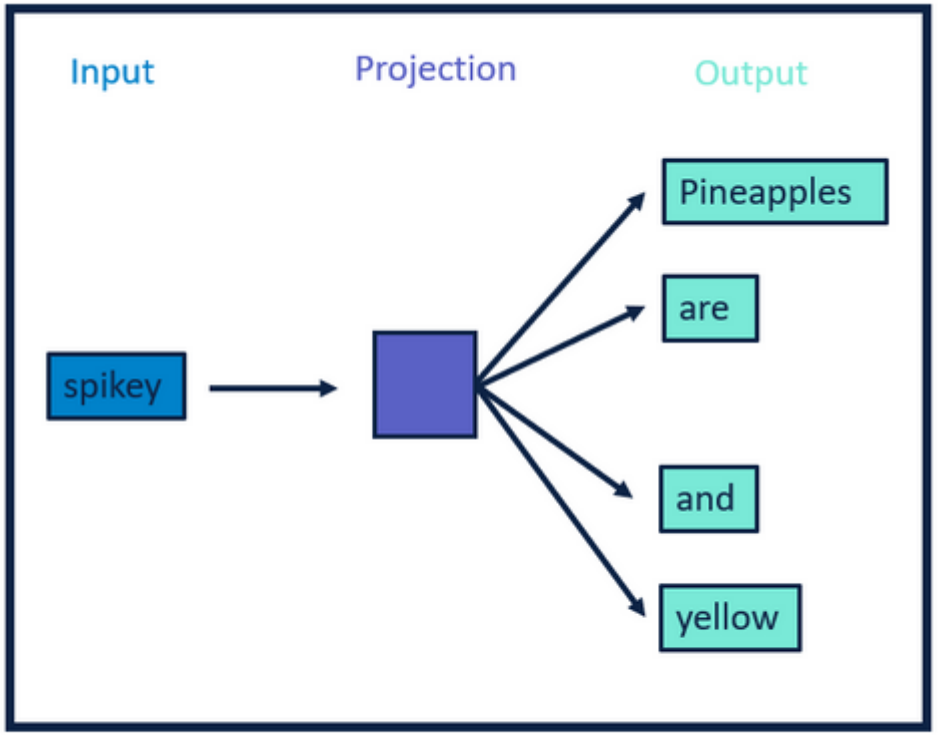
The word representations computed using neural networks are very interesting because the learned vectors explicitly encode many linguistic regularities and patterns. Somewhat surprisingly, many of these patterns can be represented as linear translations. For example, the result of a vector calculation vec("Madrid") - vec("Spain") + vec("France") is closer to vec("Paris") than to any other word vector [9, 8].

Word2Vec

- Distributed representations of words and phrases and their compositionality, Mikolov et al. NeurIPS13



CBOW



Skip-gram

Word2Vec

- Distributed representations of words and phrases and their compositionality, Mikolov et al. NeurIPS13

Description about monkey banana	Preprocessed sentence
생강목 파초과에 해당하는 바나나의 한 종류로 크기가 작고 단맛이 강하다. 원숭이들이 즐겨 먹는 종이라 하여 '몽키 바나나'라 이름 붙여졌으며 크기가 작아 '미니 바나나' 혹은 '세뇨리타 바나나(Señorita banana)'라고도 불린다. 필리핀이 원산지로 해발 700m 이상의 야생 고산지대에서 자생 및 재배된다.	생강목 파초과 바나나 크기 작다 단맛 원숭이 먹다 '몽키 바나나' 크기 작다 '미니 바나나' 필리핀

Word2Vec

- Distributed representations of words and phrases and their compositionality, Mikolov et al. NeurIPS13

생강목 파초과 바나나 크기 작다 단맛 원숭이 먹다 '몽키 바나나' 크기 작다 '미니 바나나' 필리핀



Word2Vec

- Word2Vec viewed as logistic regression

$$p(y = 1 | (w_i, w_j)) = \sigma(e_i \cdot e_j)$$

- For example,

$$p\left(y = 1 \middle| \left(w_{\text{몽키바나나}}, w_{\text{몽키}}\right)\right) = \sigma\left(e_{\text{몽키바나나}} \cdot e_{\text{몽키}}\right)$$

$$\mathcal{L}_i = -\frac{1}{|C|} \sum_{w_j \in C_i} \log(\sigma(e_i \cdot e_j))$$

Word2Vec

- This problem setting contains **only positive labels**
  - Unary classification, also known as one-class classification or single-class classification, is a type of classification problem where the goal is to identify instances belonging to a single class—the positive class
  - In unary classification, only positive examples are available during training, with no negative examples to learn from
  - The absence of negative examples *limits the model's discriminative power*, making it challenging to distinguish between truly positive instances and those that are dissimilar but were not explicitly provided as negative examples

Word2Vec

- Negative sampling is a method used to alleviate the limitations of unary classification
  - Negative samples are generated from the dictionary



Word2Vec

- Performing pairwise logistic regression
  - Pairing the samples (target, positive, negative):
    - E.g., (몽키바나나나, 바나나나, 파리), (몽키바나나나, 원숭이, 침대)



Word2Vec

- Pairwise logistic regression

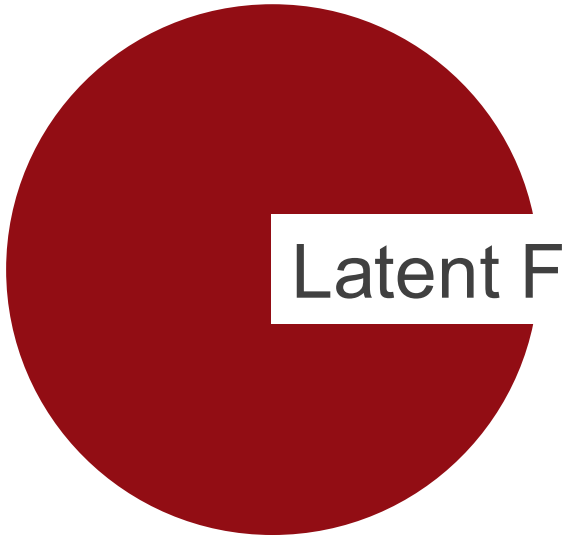
$$p\left(y = 1 \middle| \left(w_i, w_j, \bar{w}_k\right)\right) = \sigma\left(e_i \cdot e_j - e_i \cdot e_k\right)$$

- Loss function is defined as follows:

$$\mathcal{L}_i = -\frac{1}{|C|} \sum_{w_j \in C_i} \sum_{\bar{w}_k \in N_i} \log\left(\sigma\left(e_i \cdot e_j - e_i \cdot e_k\right)\right)$$

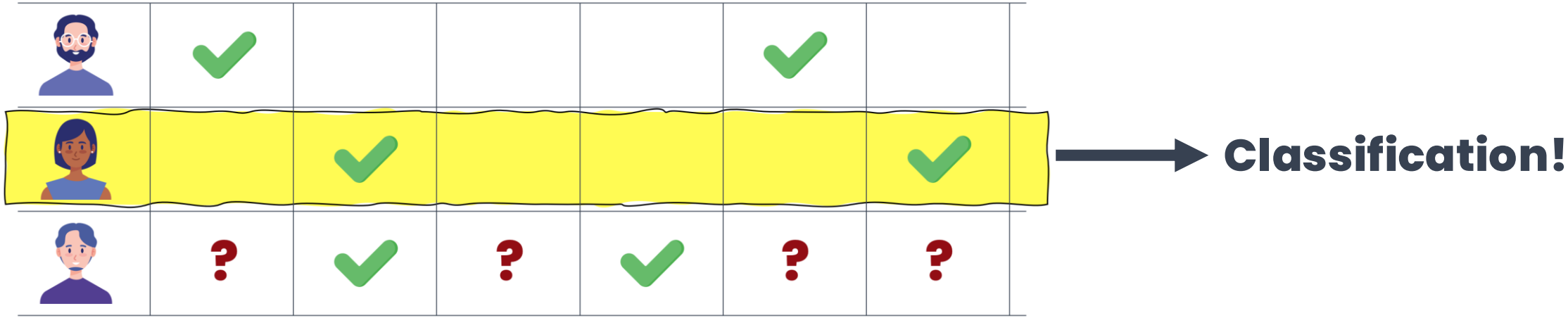
Word2Vec

- Effects of negative sampling
  - Handling Imbalanced Datasets:
    - In many real-world applications, the negative class (e.g., non-relevant items or words that don't co-occur) can be highly imbalanced compared to the positive class
    - Negative sampling *addresses this data imbalance* by enabling the model to focus on a small subset of negative examples during each training iteration
  - Faster Convergence:
    - By focusing on a subset of negative examples, negative sampling often results in *faster convergence* during training
    - This allows the model to update its parameters more quickly, which is especially beneficial when working with large datasets
  - Enhanced Generalization:
    - Negative sampling enables the model to learn from a *diverse set of negative examples*, *improving its ability to generalize to unseen data*
    - This is crucial for developing models that perform well on real-world tasks



Latent Factor Models for Implicit Feedbacks

Recap: Implicit Feedback



<Implicit Feedback>

Recap: Logistic Regression

- A simple form of rating prediction using logistic regression is defined as follows:

$$x = o_i + p_j + \sum_s^k u_{is} \cdot v_{js}$$

$$\sigma(x) = \frac{1}{1 + e^{-x}}$$

$$p(y = 1 | (u_i, v_j)) = \hat{r}_{ij} = \sigma(x) = \sigma\left(o_i + p_j + \sum_s^k u_{is} \cdot v_{js} \right)$$

Limitations of Implicit Feedback

- In recommender systems, much of the implicit feedback data contains **only positive labels**



<Implicit Feedback: Friend Recommendation>

Unary Classification

- The goal of unary classification is to identify instances that belong to a single class—the positive class
- In this setting, only positive examples are available during training, with no negative examples to learn from

- However, personal preferences cannot be fully captured by positive feedback alone
- Negative feedback on items is also valuable for understanding individual preferences

Recap: Logistic Regression: Optimization

- The logistic regression model is optimized by minimizing the following *log loss function*, rather than using mean squared error (MSE):

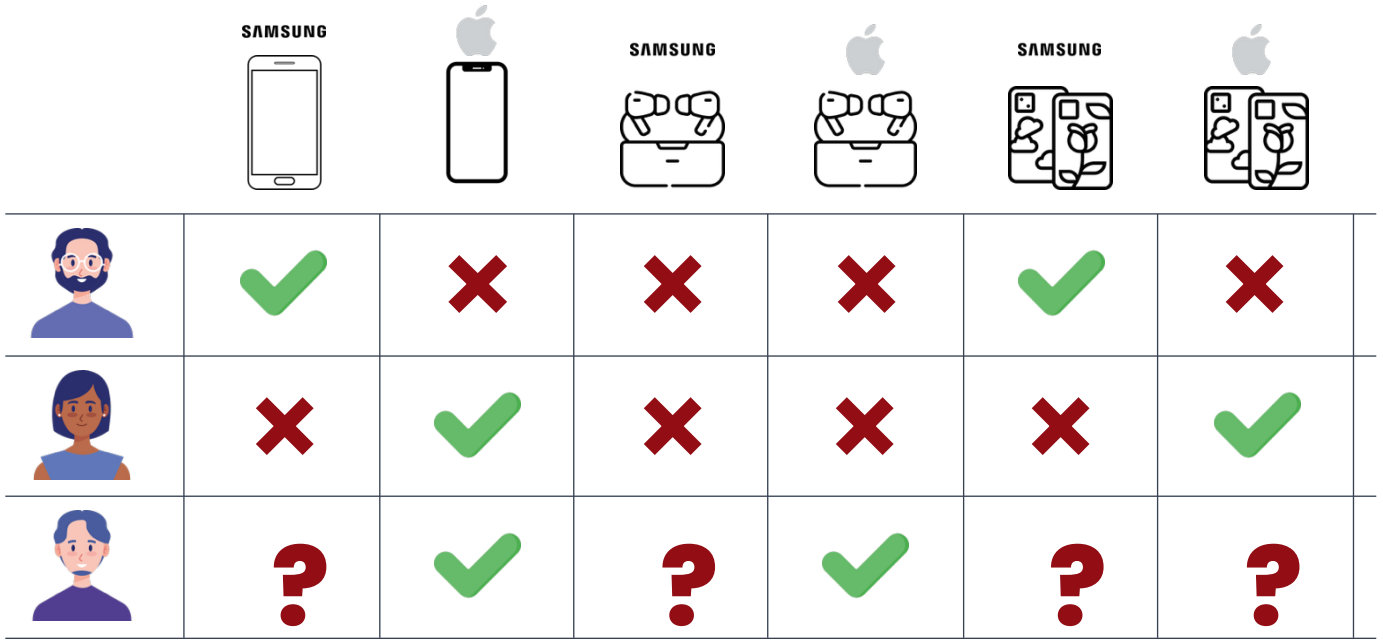
$$\mathcal{L} = -\frac{1}{|D|}\sum_{d_i \in D} y_i \log\big(p(y_i)\big) + (1 - y_i) \cdot \log\big(1 - p(y_i)\big)$$

where $d = (u, v, y)$

- In binary classification, the log loss function is equivalent to the cross-entropy loss function

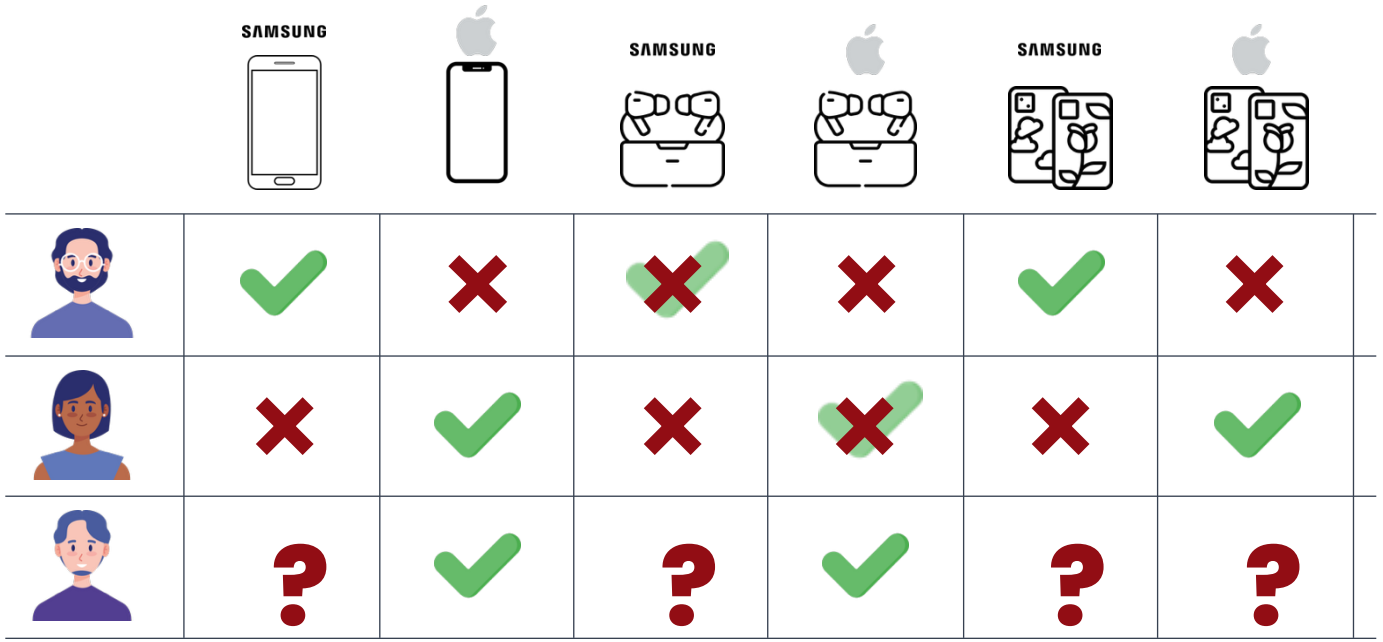
Negative Samples in Implicit Feedback

- The simplest method for creating negative samples is to assign negative labels to all items with which the user has not interacted



Negative Samples in Implicit Feedback

- However, this straightforward approach carries the risk of misclassifying potential positive interactions as negative feedback



Negative Sampling for Latent Factor Model

- However, this straightforward approach carries the risk of misclassifying potential positive interactions as negative feedback

- To address this issue, ***negative sampling*** is commonly used in recommendation systems
- Instead of treating all non-interactive items as negative samples, a few are randomly selected to perform logistic regression

Logistic Regression with Negative Samples

- A simple form of logistic regression using negative samples is described as follows:

$$x = u_i \cdot v_j - u_i \cdot v_k$$

$$\sigma(x) = \frac{1}{1 + e^{-x}}$$

$$p\left(y = 1 \middle| \left(u_i, v_j, v_k\right)\right) = \sigma(x) = \sigma\left(u_i \cdot v_j - u_i \cdot v_k\right)$$

- Therefore, the loss function for logistic regression is defined as follows:

$$\mathcal{L} = -\frac{1}{|N_i|} \sum_{v_k \in N_i} \log\left(\sigma\left(u_i \cdot v_j - u_i \cdot v_k\right)\right)$$

Logistic Regression with Negative Feedback and Negative Samples

- If implicit feedback includes actual negative feedback rather than unary data, how can negative sampling still be utilized?

- We can simply prioritize and pair positive feedback, negative feedback, and unobserved negative samples according to the given priority:

$$positive\ feedback\ v^{+} > negative\ sample\ v^{*} > negative\ feedback\ v^{-}$$

슬라이드 수정 필요

Logistic Regression with Negative Feedback and Negative Samples (cont'd)

- For a user u_i and an item v_j with positive feedback, we can form the following tuples:

$$(u_i, v_j, v^*), (u_i, v_j, v^-), (u_i, v^*, v^-), \qquad v_j \in I_i^+, \qquad v^* \in N_i, \qquad v^- \in I_i^-$$

$$\begin{aligned}
&\mathcal{L} \\
&= -\left(\frac{1}{|I_i^+| \cdot |N_i|} \sum_{v_j \in I_i^+} \sum_{v_k \in N_i} \log\left(\sigma(e_i \cdot e_j - e_i \cdot e_k)\right) + \frac{1}{|I_i^+| \cdot |I_i^-|} \sum_{v_j \in I_i^+} \sum_{v_k \in I_i^-} \log\left(\sigma(e_i \cdot e_j - e_i \cdot e_k)\right) \right. \\
&\left. + \frac{1}{|N_i| \cdot |I_i^-|} \sum_{v_j \in N_i} \sum_{v_k \in I_i^-} \log\left(\sigma(e_i \cdot e_j - e_i \cdot e_k)\right) \right)
\end{aligned}$$